SOFT COMPUTING AND APPLICATIONS (CSE320)

PROJECT REPORT

on

SMART FAN SPEED CONTROLLER: A COMPARATIVE STUDY AND REAL-TIME SIMULATION USING CRISP THRESHOLDS, MAMDANI FUZZY INFERENCE, AND BACKPROPAGATION NEURAL NETWORK

Submitted in partial fulfilment of the requirements of the course
Soft Computing and Applications (CSE320)
for the degree of

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE ENGINEERING

Submitted by:
ANSHUMAAN SINGH
Enrollment No.: A023167023097
Section: 7-DS-2(X)

Course Instructor:
Dr. Smriti Sehgal

AMITY SCHOOL OF ENGINEERING AND TECHNOLOGY (ASET)
AMITY UNIVERSITY, NOIDA
November 2026

GitHub Repository: https://github.com/Anshumaan-25/Smart-Fan-Speed-Controller

DECLARATION

I, **Anshumaan Singh**, student of Bachelor of Technology in Computer Science Engineering at **Amity School of Engineering and Technology (ASET), Amity University, Noida**, hereby declare that the work presented in this project report, titled **"Smart Fan Speed Controller: A Comparative Study and Real-Time Simulation Using Crisp Thresholds, Mamdani Fuzzy Inference, and Backpropagation Neural Network,"** is an authentic record of original engineering work carried out by me for the course **Soft Computing and Applications (CSE320)** under the supervision and guidance of **Dr. Smriti Sehgal**.

I further declare that this report has not been submitted previously, in part or in full, to any other university, institute, or examining body for the award of any degree, diploma, fellowship, or other academic recognition. All sources of theoretical literature, algorithms, and benchmark references have been duly cited and acknowledged.

_____

Anshumaan Singh
Enrollment No.: A023167023097
Section: 7-DS-2(X)

Place: Noida
Date: 11 November 2026

ACKNOWLEDGEMENT

I would like to express my deepest gratitude and sincere appreciation to my course instructor, **Dr. Smriti Sehgal**, for her invaluable lectures, technical guidance, constructive feedback, and continuous encouragement throughout the course **Soft Computing and Applications (CSE320)**. Her pedagogy in explaining the mathematical foundations of classical logic, fuzzy systems, artificial neural architectures, and learning algorithms provided the necessary clarity to conceptualize and execute this project from foundational first principles.

I am profoundly grateful to the Department of Computer Science and Engineering and the administration of **Amity School of Engineering and Technology (ASET), Amity University, Noida**, for offering an exceptional academic infrastructure, computational facilities, and learning ecosystem that fostered the development and testing of this software engineering project.

Finally, I extend my heartfelt thanks to my family, peers, and fellow classmates of Section 7-DS-2(X) for their constant discussions, support, and collaborative academic spirit.

Anshumaan Singh
Enrollment No.: A023167023097
B.Tech CSE, Section 7-DS-2(X)

ABSTRACT

This project report presents an end-to-end design, mathematical modeling, implementation, and empirical comparison of three distinct computational control paradigms applied to an automated room fan speed control system based on environmental ambient temperature ($18°C \leq T \leq 42°C$) and relative humidity ($0\% \leq H \leq 100\%$). The project was undertaken for the course **Soft Computing and Applications (CSE320)** under the guidance of **Dr. Smriti Sehgal**.

Soft Computing encompasses computational techniques—principally fuzzy logic, artificial neural networks, and evolutionary computation—that exploit the tolerance for imprecision, uncertainty, approximate reasoning, and partial truth to achieve tractability, robustness, and low solution cost. In contrast to conventional hard (crisp) computing which enforces rigid binary decision thresholds, soft computing methodologies enable continuous, non-linear, and adaptive responses.

To demonstrate these principles empirically, three independent controllers were developed using **pure Python standard library** primitives (without third-party mathematical or machine learning packages):

1. **Module 1 · Crisp Threshold Controller:** Models rigid industrial thermostat logic with discrete threshold jumps at 24°C and 30°C and a step humidity offset at 70% RH.
2. **Module 2 · Multilayer Perceptron (MLP) Neural Network:** A 2–6–1 feedforward architecture with hyperbolic tangent (&tanh;) hidden units trained from scratch via analytical Backpropagation and Stochastic Gradient Descent (SGD) across 100 epochs under two learning rates ($\eta = 0.01$ and $\eta = 0.08$).
3. **Module 3 · Mamdani Fuzzy Inference System:** Utilizes continuous triangular and shoulder membership functions forming a strict partition of unity, a 9-rule linguistic matrix, Mamdani min-implication, max-aggregation over 5 output fuzzy sets, and discrete Centroid defuzzification.

All three controllers were benchmarked against a continuous synthetic comfort policy featuring non-linear thermal-humidity interaction dynamics. Evaluation was conducted on a strictly segregated held-out test grid of 120 midpoint conditions disjoint from the 143 training points. The analytical backpropagation engine was formally verified against numerical finite-difference quotients ($\varepsilon = 10^{-6}$) to 8 decimal places. Experimental findings reveal that while the crisp controller produces severe boundary discontinuities and chattering (Mean Absolute Error of 13.79 percentage points [pp]), the Mamdani fuzzy controller delivers smooth heuristic actuation without requiring any numerical training data (9.96 pp MAE), and the backpropagation neural network successfully approximates the continuous target manifold with sub-percentage point precision (0.94 pp MAE at $\eta = 0.08$).

An interactive desktop simulation application was built in Python's Tkinter/Canvas framework, featuring real-time vector blade animation, parametric sweep plotting, dynamic fuzzy rule matrix telemetry, and logarithmic loss trajectory tracking. The full source code, regression tests, and documentation are hosted on GitHub at: https://github.com/Anshumaan-25/Smart-Fan-Speed-Controller.

Contents

1 Introduction **7**

1.1 Introduction 7

1.2 Background and Motivation 7

1.3 Course Context and Objectives 7

1.4 Problem Statement and Project Objectives 8

1.5 Overview of the Proposed Solution and Report Structure 8

2 Soft Computing and Theoretical Background **9**

2.1 Introduction 9

2.2 Soft Computing and its Constituents 9

2.3 Hard Computing (Crisp Sets) vs. Soft Computing 9

2.4 Artificial Neural Networks and Supervised Learning 10

2.5 Multilayer Perceptrons and Analytical Backpropagation 10

2.6 Fuzzy Sets, Membership Functions, and Partition of Unity 12

2.7 Fuzzy Rule Base, Mamdani Inference, and Defuzzification 12

2.8 Concept Mapping: Soft Computing Principles in This Project 13

3 Experimental Setup and Work Environment **14**

3.1 Methodology 14

3.2 Software, Hardware and Technology Stack 14

3.3 Version Control and GitHub Repository 14

3.4 Project Milestones 15

4 The Project: Smart Fan Speed Controller **16**

4.1 Overview, Motivation and Problem Statement 16

4.2 Objectives, Operational Scope and Constraints 16

4.3 Input Domain, Normalization and Synthetic Target Formulation 16

5 Project Methodology and Implementation **18**

5.1 Environment Setup and Architecture Partitioning 18

5.2 Module 1: Crisp Threshold Controller Implementation 18

5.3 Module 2: Multilayer Perceptron & Backpropagation Implementation 18

5.4 Module 3: Mamdani Fuzzy Inference Engine Implementation 20

5.5 Dataset Generation and Disjoint Data Segregation 21

5.6 Interactive Desktop Simulator Development (`fan_lab.py`) 22

5.7 Automated Regression Test Suite (`test_controllers.py`) 22

6 Results and Discussion **24**

6.1 Dataset Composition and Partitioning Analysis 24

6.2 Neural Network Training Convergence and Loss Analysis 24
6.3 Generalization Benchmark on Held-Out Midpoints 25
6.4 Boundary Discontinuity and Chattering in Crisp Control 26
6.5 Fuzzy Interpolation Characteristics and Centroid Stability 26
6.6 Numerical Gradient Verification Analysis 27
6.7 Overall Discussion, Trade-Offs, and Limitations 27
7 Challenges Faced and Solutions **28**
8 Relevance to Soft Computing and Learning Outcomes **29**
9 Conclusion and Future Scope **30**
9.1 Conclusion 30
9.2 Future Scope 30
References **31**
Appendices **32**
A. Algorithm Core Implementation (`controllers.py`) 32
B. Interactive Desktop GUI (`fan_lab.py`) 36
C. Automated Unit Test Suite (`test_controllers.py`) 46
D. GitHub Repository Structure and Verification 48

1 Introduction

1.1 Introduction

Modern environmental control systems, such as smart Heating, Ventilation, and Air Conditioning (HVAC) units and intelligent cooling fans, are responsible for maintaining thermal comfort within indoor living and working environments. The challenge of fan speed regulation lies in translating continuous, correlated environmental telemetry—primarily temperature and humidity—into smooth, energy-efficient, and wear-free mechanical actuation. In conventional control engineering, systems rely heavily on rigid threshold logic (hard computing). However, real-world physical environments are characterized by graded transitions, non-linear physiological comfort interactions, and imprecise sensory readings. This project investigates and demonstrates how soft computing techniques—specifically fuzzy inference systems and artificial neural networks—resolve the fundamental shortcomings of classical crisp control systems.

1.2 Background and Motivation

Conventional thermostat and fan regulators operate on step-wise thresholds. For instance, a basic multi-speed fan switch may engage "Low" speed below 24°C, "Medium" speed from 24°C to 30°C, and "High" speed above 30°C. While simple to implement, such hard-switching policies suffer from severe operational pathologies:

- **Boundary Discontinuity:** A microscopic temperature fluctuation (e.g., from 29.9°C to 30.1°C) produces a sudden, large jump in rotational velocity (e.g., from 50% to 75% power).
- **Actuator Hunting and Chattering:** When ambient conditions hover near a decision boundary, thermal turbulence causes rapid oscillations between operating states, causing severe mechanical wear on bearings and motor coils, electrical contact degradation, and audible noise pollution.
- **Inadequate Multivariable Synthesis:** Humidity dramatically alters human perceived temperature through the heat index. In a crisp system, factoring in relative humidity requires branching nested conditional statements that yield complex, discontinuous decision surfaces.

These limitations directly motivate the application of **Soft Computing** paradigms that inherently model gradual transitions, smooth interpolations, and non-linear data manifolds.

1.3 Course Context and Objectives

This project was developed for the curriculum of **Soft Computing and Applications (CSE320)** at Amity School of Engineering and Technology (ASET), Amity University, Noida, under the faculty supervision of **Dr. Smriti Sehgal**. The course syllabus explores three primary foundational modules:

- **Module 1 · Crisp Logic and Classical Set Theory:** Discrete membership functions, Boolean operations, sharp decision boundaries, and crisp rule structures.
- **Module 2 · Artificial Neural Networks (ANN):** Biological neural analogies, artificial neuron models, multi-layer feedforward networks, activation functions, error backpropagation, and gradient-based weight optimization.
- **Module 3 · Fuzzy Logic Systems:** Fuzzy sets, continuous membership functions, linguistic variables, approximate reasoning, fuzzy rule bases, fuzzy implication, aggregation, and defuzzification techniques.

The project synthesizes these three modules into an integrated engineering simulator that directly compares crisp logic, backpropagation neural networks, and Mamdani fuzzy systems under identical environmental conditions.

1.4 Problem Statement and Project Objectives

Problem Statement: Given ambient room temperature $T \in [18, 42]$°C and relative humidity $H \in [0, 100]$%, design, mathematically formulate, implement, and benchmark three independent control paradigms to compute an optimal continuous fan speed $S \in [0, 100]$%, while analyzing boundary stability, interpolation accuracy, and algorithmic complexity.

The core engineering objectives are:

1. Implement the mathematical foundations of all three controllers in pure standard-library Python without external numerical dependencies.
2. Formulate a synthetic comfort target policy containing non-linear interaction terms to serve as an objective evaluation benchmark.
3. Derive and implement analytical backpropagation for a 2–6–1 Multilayer Perceptron, verifying weight gradients against numerical finite-difference approximations.
4. Design a complete Mamdani fuzzy inference engine with partition-of-unity membership functions and exact centroid defuzzification.
5. Benchmark all models across a strictly disjoint held-out test grid of 120 midpoints using Mean Absolute Error (MAE).
6. Develop an interactive desktop demonstration application (`fan_lab.py`) with real-time vector graphics and analytical telemetry.
7. Publish the verified codebase to GitHub under version control at https://github.com/Anshumaan-25/Smart-Fan-Speed-Controller.

1.5 Overview of the Proposed Solution and Report Structure

The proposed solution implements a clean separation of concerns: mathematical algorithms and benchmark pipelines are encapsulated in `controllers.py`, interactive visualization and GUI state machines are built in `fan_lab.py`, and regression tests are implemented in `test_controllers.py`.

The remainder of this report is structured as follows: Chapter 2 reviews the theoretical foundations of soft computing. Chapter 3 outlines the experimental environment, toolchain, and GitHub setup. Chapter 4 details the problem domain and target formulation. Chapter 5 documents the implementation methodology. Chapter 6 presents quantitative results and comparative discussion. Chapter 7 summarizes challenges encountered and engineering solutions. Chapter 8 highlights curriculum learning outcomes. Chapter 9 concludes the study and outlines future enhancements. Complete source listings and repository details are provided in the Appendices.

2 Soft Computing and Theoretical Background

2.1 Introduction

This chapter establishes the theoretical foundations of the computational paradigms utilized in this project. It contrasts hard computing with soft computing, develops the mathematical basis of Artificial Neural Networks and analytical backpropagation, and details the calculus of Fuzzy Logic Systems.

2.2 Soft Computing and its Constituents

The concept of Soft Computing, introduced by Lotfi A. Zadeh in the early 1990s, represents a significant paradigm shift from conventional analytical and algorithmic computing. Traditional computing paradigms demand precise inputs, exact mathematical models, and unambiguous true/false Boolean states. In contrast, soft computing is founded on the premise that real-world phenomena are inherently imprecise, uncertain, and partially true.

The guiding principle of soft computing is: *"Exploit the tolerance for imprecision, uncertainty, partial truth, and approximation to achieve tractability, robustness, and low solution cost."*

The principal constituents of soft computing are:

- **Fuzzy Logic (FL):** Provides a mathematical framework for approximate reasoning and linguistic knowledge representation.
- **Artificial Neural Networks (ANN):** Provides data-driven learning, non-linear pattern recognition, and adaptive function approximation.
- **Evolutionary Computation (EC):** Provides stochastic global search and parameter optimization (e.g., Genetic Algorithms).

2.3 Hard Computing (Crisp Sets) vs. Soft Computing

In hard computing, set membership is binary and governed by the characteristic indicator function χ_A:

$$\chi_A(x) \in \{0, 1\}$$

An element x either strictly belongs to set A ($\chi = 1$) or does not ($\chi = 0$). This binary rigidity underlies crisp threshold controllers. In contrast, soft computing replaces the binary indicator function with a continuous membership function μ_A:

$$\mu_A(x) \in [0, 1]$$

This continuous grading allows an ambient temperature of 23.9°C to belong simultaneously to the set "Cool" with degree 0.02 and to "Warm" with degree 0.98, eliminating sharp transitions.

Dimension	Hard Computing (Crisp Systems)	Soft Computing (Fuzzy & Neural Systems)
Underlying Logic	Two-valued Aristotelian Boolean logic (True / False)	Multi-valued continuous logic & graded truth values

Tolerance for Imprecision	Zero; requires exact mathematical formulations	High; tolerant of noise, imprecision, and uncertainty
Boundary Transitions	Sharp, discontinuous step boundaries	Smooth, continuous, asymptotic transitions
Knowledge Representation	Rigid lookup tables, hard IF-THEN conditionals	Linguistic rules, distributed synaptic weight matrices
Adaptability	Static; requires manual code refactoring	Adaptive; learns from data or adjusts through tuning

2.4 Artificial Neural Networks and Supervised Learning

Artificial Neural Networks are computational systems inspired by the structure and distributed processing capabilities of biological neural networks. An artificial neuron computes a weighted sum of its inputs, adds an internal bias parameter, and passes the resulting scalar activation through a non-linear activation function.

In supervised learning, the network is presented with paired training exemplars $\{(\mathbf{x}_k, y_k)\}$. The learning objective is to optimize the network's internal parameter tensors (synaptic weights **W** and biases **b**) to minimize an empirical loss function over the training corpus:

$$L(\boldsymbol{W}, \boldsymbol{b}) = (1 / 2N) \sum || f(\boldsymbol{x}_k; \boldsymbol{W}, \boldsymbol{b}) - y_k ||^2$$

2.5 Multilayer Perceptrons and Analytical Backpropagation

While a single-layer perceptron can only resolve linearly separable classification boundaries, a Multilayer Perceptron (MLP) containing one or more non-linear hidden layers acts as a universal function approximator (Cybenko, 1989).

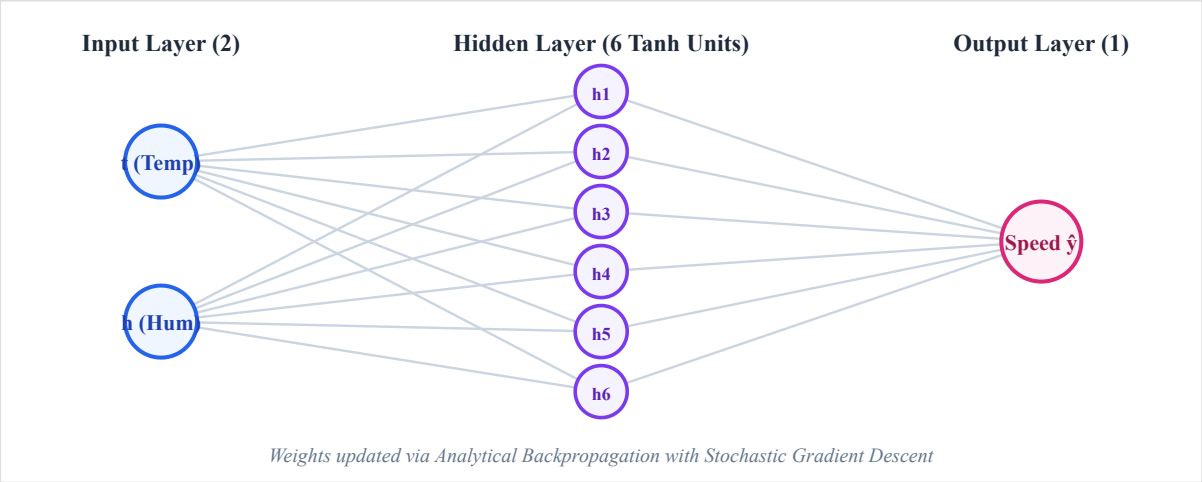


Figure 2.1: Mathematical topology of the 2–6–1 Multilayer Perceptron implemented in Module 2.

The network topology implemented in this project consists of:

- **Input Layer:** 2 neurons representing normalized temperature t and humidity h.
- **Hidden Layer:** 6 hidden units utilizing the hyperbolic tangent activation function:

$$\&tanh;(z) = (e^z - e^{-z}) / (e^z + e^{-z})$$

- **Output Layer:** 1 linear output neuron computing normalized fan speed.

Analytical backpropagation calculates the exact gradient of the loss function with respect to every weight parameter by systematic application of the calculus chain rule. For the hidden layer, the derivative of &tanh;(z) is compactly evaluated in terms of its output activation:

$$d/dz\ [\&tanh;(z)] = 1 - \&tanh;^{2}(z) = 1 - h_j^2$$

This mathematical property eliminates the need for expensive transcendental evaluations during the backward error pass.

2.6 Fuzzy Sets, Membership Functions, and Partition of Unity

A fuzzy set A in universe of discourse U is formally characterized by a continuous membership function μ_A: U → [0, 1]. In this project, the normalized input domains t ∈ [0, 1] and h ∈ [0, 1] are partitioned into three linguistic terms:

- **Low / Cool / Dry:** Left-shoulder membership function:

$$\mu_{Low}(x) = max(0,\ 1 - 2x)$$

- **Medium / Warm / Comfortable:** Symmetric triangular membership function:

$$\mu_{Med}(x) = 1 - |2x - 1|$$

- **High / Hot / Humid:** Right-shoulder membership function:

$$\mu_{High}(x) = max(0,\ 2x - 1)$$

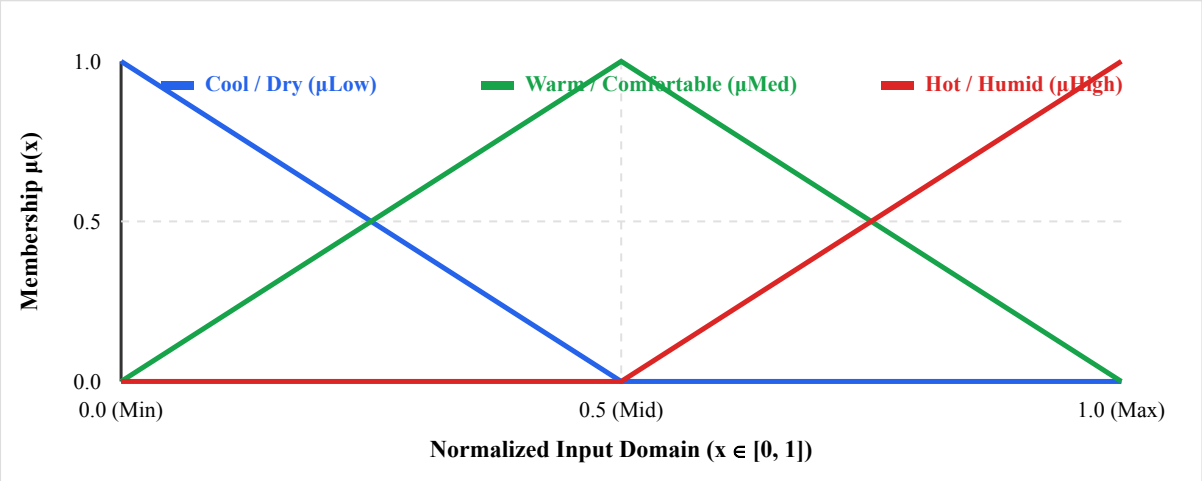


Figure 2.2: Continuous triangular and shoulder input membership functions forming a partition of unity.

Theorem: Partition of Unity
A fuzzy partitioning forms a strict *partition of unity* if and only if the sum of all membership values at any point in the universe equals exactly one:

$$\sum_i \mu_i(x) = 1,\ \forall x \in [0,\ 1]$$

Proof for the implemented system:
Case 1: For x ∈ [0, 0.5]:
$\mu_{Low}(x) + \mu_{Med}(x) + \mu_{High}(x) = (1 - 2x) + (1 - (1 - 2x)) + 0 = (1 - 2x) + 2x = 1.0$.
Case 2: For x ∈ [0.5, 1.0]:
$\mu_{Low}(x) + \mu_{Med}(x) + \mu_{High}(x) = 0 + (1 - (2x - 1)) + (2x - 1) = (2 - 2x) + (2x - 1) = 1.0$.
Hence, $\sum \mu_i(x) = 1$ identically across the entire domain, guaranteeing seamless interpolation without dead zones.

2.7 Fuzzy Rule Base, Mamdani Inference, and Defuzzification

A Mamdani Fuzzy Inference System executes four sequential stages:

1. **Fuzzification:** Continuous crisp inputs (t, h) are mapped to membership degrees across linguistic sets.
2. **Rule Evaluation (T-norm Implication):** For each multi-antecedent rule "IF Temperature is A_i AND Humidity is B_j THEN Speed is C_k", the rule firing strength $\alpha_{i,j}$ is determined using the Zadeh intersection (minimum operator):

$$\alpha_{i,j} = min(\mu_{A_i}(t), \mu_{B_j}(h))$$

3. **Aggregation (S-norm):** The qualified consequent fuzzy sets are unified using the maximum operator across all rules:

$$\mu_{agg}(s) = max_k \left[min(\beta_k, \mu_{C_k}(s)) \right]$$

4. **Defuzzification (Centroid Method):** Converts the aggregated fuzzy distribution into a single crisp scalar actuator value representing the center of gravity (centroid):

$$S_{fuzzy} = \sum_{s=0}^{100} s \cdot \mu_{agg}(s) / \sum_{s=0}^{100} \mu_{agg}(s)$$

2.8 Concept Mapping: Soft Computing Principles in This Project

The table below documents how foundational theoretical concepts from Modules 1–3 of CSE320 are mapped directly to concrete algorithmic implementations in this project.

Soft Computing Concept	Curriculum Module	Implementation in This Project
Crisp Set Indicator Functions	Module 01	Hard threshold boundaries in `crisp_speed(T, H)` at 24°C, 30°C, and 70% RH.
Non-linear Activation Functions	Module 02	Hyperbolic tangent (`math.tanh`) applied across 6 hidden units in `NeuralController`.
Supervised Gradient Learning	Module 02	Stochastic Gradient Descent (SGD) minimizing half squared error over 100 epochs.
Analytical Backpropagation	Module 02	Exact chain-rule derivation propagating output delta through hidden weights and $(1 - h^2)$.
Gradient Verification	Module 02	Automated numerical two-sided finite-difference gradient check (`test_controllers.py`).
Fuzzification & Partition of Unity	Module 03	Triangular and shoulder membership calculus in `memberships(value)` satisfying $\sum \mu = 1$.
T-norm Conjunction (Mamdani Min)	Module 03	Antecedent rule evaluation via `min(heat_degree, moisture_degree)` across 9 rules.
S-norm Aggregation (Mamdani Max)	Module 03	Consequent aggregation via `max(min(strength, degree))` across 5 triangular output sets.
Centroid Defuzzification	Module 03	Discrete center of gravity summation evaluated across integer speeds $s \in \{0, ..., 100\}$.

3 Experimental Setup and Work Environment

3.1 Methodology

The project followed an incremental and disciplined engineering methodology:

1. **Mathematical Modeling:** Formulate closed-form mathematical equations for the reference comfort manifold, crisp step rules, backpropagation updates, and fuzzy implication.
2. **Pure Standard-Library Development:** Implement core algorithms in `controllers.py` without external dependencies to maintain transparency and platform portability.
3. **Automated Test-Driven Verification:** Construct regression unit tests in `test_controllers.py` to verify threshold edge cases, partition disjointness, and analytical gradient correctness.
4. **Interactive Simulator Construction:** Develop a multi-panel desktop GUI in `fan_lab.py` using Python's standard `tkinter` framework with vector canvas rendering.
5. **Segregated Generalization Benchmarking:** Evaluate interpolation errors across disjoint held-out test midpoints.
6. **Version Control and Cloud Deployment:** Initialize Git, structure the repository, and publish to GitHub.

3.2 Software, Hardware and Technology Stack

Development and testing were conducted on a macOS workstation running native Python 3. The technical specifications of the development environment are summarized below.

Component	Specification / Detail
Hardware Workstation	Apple Silicon Workstation (macOS Sonoma / Darwin Kernel)
Programming Language	Python 3.13 / 3.9+ (Standard Library: `math`, `random`, `dataclasses`, `unittest`, `tkinter`)
GUI Framework	Tkinter / TTK (Tk Version 8.6+ with custom double-buffered Canvas rendering)
Version Control	Git 2.45+ & GitHub Cloud Remote
Documentation Toolchain	Pandoc 3.10, Headless Google Chrome, PyPDF
External Packages Required	**None (Zero third-party dependencies)**

3.3 Version Control and GitHub Repository

The complete project repository is managed under Git version control and hosted publicly on GitHub.

Property	Value / Link
Repository URL	https://github.com/Anshumaan-25/Smart-Fan-Speed-Controller
Author / GitHub Handle	Anshumaan Singh (`Anshumaan-25`)

Default Branch	`main`
Key Files in Repository	`controllers.py`, `fan_lab.py`, `test_controllers.py`, `Launch-Fan-Lab.command`, `README.md`

3.4 Project Milestones

The project execution was organized into eight structured milestones (M1 through M8), ensuring that each phase was verified before proceeding to the next.

Milestone	Description	Status
M1	Problem definition, reference policy formulation, and input normalization calculus	Completed
M2	Module 1 Crisp Threshold Controller implementation and edge-case testing	Completed
M3	Module 2 Multilayer Perceptron architecture, SGD, and analytical backpropagation engine	Completed
M4	Numerical gradient verification check ($\varepsilon = 10^{-6}$) matching backprop deltas	Completed
M5	Module 3 Mamdani Fuzzy Inference Engine with partition-of-unity and centroid defuzzification	Completed
M6	Held-out midpoint grid benchmark generation (143 train vs 120 test cases)	Completed
M7	Interactive desktop GUI simulator (`fan_lab.py`) with animated vector fan and sweep charts	Completed
M8	GitHub cloud repository publishing, test validation, and formal academic report generation	Completed

4 The Project: Smart Fan Speed Controller

4.1 Overview, Motivation and Problem Statement

The Smart Fan Speed Controller is an environmental actuator control system designed to regulate mechanical cooling fan velocity based on continuous telemetry of ambient room temperature and relative humidity. Indoor thermal comfort is governed by biophysical heat transfer dynamics. Human skin perceives heat dissipation not solely as a function of dry-bulb air temperature, but through an evaporative cooling rate strongly governed by ambient moisture (relative humidity). In hot and humid conditions, sweat evaporation is hindered, necessitating higher convective airflow. Conversely, in cold or dry environments, excessive airflow causes draft discomfort and hypothermia risks.

The engineering problem requires designing an automated controller capable of translating any arbitrary temperature-humidity coordinate within the operational envelope into an optimal percentage fan speed, ensuring smooth actuation without sudden speed surges.

4.2 Objectives, Operational Scope and Constraints

The operational envelope and constraints governing the controller are:

- **Operational Temperature Range:** $T \in [18.0, 42.0]$°C. Readings outside this range represent sensor faults or extreme conditions and are rejected.
- **Operational Humidity Range:** $H \in [0, 100]$% Relative Humidity.
- **Actuator Output Envelope:** $S \in [0.0, 100.0]$% continuous rotational speed.
- **Zero External Package Constraint:** The algorithms must run on any standard Python 3 runtime without requiring `numpy`, `scipy`, or machine learning frameworks.
- **Actuator Preservation:** The controller must minimize step discontinuities that produce physical motor wear and electrical noise.

4.3 Input Domain, Normalization and Synthetic Target Formulation

To ensure uniform numeric conditioning for neural backpropagation and fuzzy inference, raw physical telemetry is normalized onto the unit hypercube $[0, 1] \times [0, 1]$:

$$t = (T - T_{min}) / (T_{max} - T_{min}) = (T - 18) / 24, \quad t \in [0, 1]$$

$$h = H / 100, \quad h \in [0, 1]$$

In the absence of expensive physical wind-tunnel telemetry, an analytical ground-truth reference policy was formulated based on human biophysical comfort dynamics:

$$Speed_{ref}(t, h) = 100 \times (0.05 + 0.63\, t + 0.12\, h + 0.16\, t\, h)$$

Key properties of this reference target function:

- **Base Convection Baseline:** At minimal conditions (T = 18°C, H = 0% &implies; t = 0, h = 0), $Speed_{ref} = 5.0\%$, maintaining gentle air circulation.
- **Maximum Cooling:** At peak conditions (T = 42°C, H = 100% &implies; t = 1, h = 1), $Speed_{ref} = 100 \times (0.05 + 0.63 + 0.12 + 0.16) = 96.0\%$.

- **Coupled Thermal-Humidity Interaction:** The bilinear cross-term 0.16 t h represents the physical phenomenon where humidity plays an increasingly aggressive role at elevated temperatures. Specifically, the partial derivative with respect to humidity:

$$\partial Speed_{ref} / \partial h = 100 \times (0.12 + 0.16\, t)$$

increases monotonically from 12.0 pp/unit-h at 18°C (t=0) up to 28.0 pp/unit-h at 42°C (t=1). This non-linear interaction tests the approximation capacity of all three controllers.

5 Project Methodology and Implementation

5.1 Environment Setup and Architecture Partitioning

The project architecture strictly separates numerical computation, user interface presentation, and verification testing:

- **`controllers.py`:** Pure standard-library module containing physical normalization, reference target model, Crisp controller, Mamdani Fuzzy Inference engine, NeuralController MLP class with backpropagation, and experiment benchmarking harness.
- **`fan_lab.py`:** Tkinter graphical demonstration application containing coordinate mapping systems (`Axes`), animated vector canvas graphics, real-time sweep plots, interactive rule grids, and logarithmic loss curves.
- **`test_controllers.py`:** Unit testing suite containing regression checks for boundary transitions, input rejection, fuzzy aggregation invariants, and finite-difference gradient checks.

5.2 Module 1: Crisp Threshold Controller Implementation

The crisp controller (`crisp_speed`) implements standard industrial thermostat threshold logic:

```
def crisp_speed(temperature, humidity):
    normalize(temperature, humidity)
    warm, hot = TEMP_THRESHOLDS  # (24, 30)
    base = 20 if temperature < warm else 50 if temperature < hot else 75
    return min(100, base + (15 if humidity >= 70 else 0))
```

Any temperature below 24°C outputs 20% base speed. Between 24°C and 30°C, base speed steps to 50%. At or above 30°C, base speed steps to 75%. If relative humidity reaches or exceeds 70%, a hard boost of +15% is added, clamped at 100%.

5.3 Module 2: Multilayer Perceptron & Backpropagation Implementation

The neural controller is implemented in the `NeuralController` class. Synaptic weights are initialized deterministically using Python's `random.Random(21)` over the uniform interval [-0.5, 0.5]:

```
class NeuralController:
    def __init__(self):
        random = Random(21)
        self.hidden = [[random.uniform(-0.5, 0.5) for _ in range(3)]
                       for _ in range(HIDDEN_UNITS)]  # 6 units, 2 inputs + bias
        self.output = [random.uniform(-0.5, 0.5) for _ in range(HIDDEN_UNITS)]
        self.bias = 0.5
```

Forward Pass Execution

Given normalized inputs t and h, the forward pass evaluates net activations and applies &tanh;:

```
    def forward(self, heat, moisture):
        hidden = [tanh(w1 * heat + w2 * moisture + bias)
                  for w1, w2, bias in self.hidden]
        prediction = self.bias + sum(weight * value
                                     for weight, value in zip(self.output, hidden))
        return hidden, prediction
```

Backpropagation Learning Engine

Training minimizes the half squared error objective E = 0.5 × (prediction - target)^2. The stochastic gradient descent loop executes across 100 epochs with randomized sample order (`Random(42)`):

```
    def train(self, samples, learning_rate):
        order = list(samples)
        history = [self.mean_squared_error(samples)]
        for _ in range(EPOCHS):
            random.shuffle(order)
            for heat, moisture, target in order:
                hidden, prediction = self.forward(heat, moisture)
                output_delta = prediction - target  # dE/d(pred)
                for index, value in enumerate(hidden):
                    # Hidden error: delta_out * w_out * (1 - tanh^2)
                    hidden_delta = output_delta * self.output[index] * (1 - value**2)
                    weights = self.hidden[index]
                    weights[0] -= learning_rate * hidden_delta * heat
                    weights[1] -= learning_rate * hidden_delta * moisture
                    weights[2] -= learning_rate * hidden_delta
                    self.output[index] -= learning_rate * output_delta * value
                self.bias -= learning_rate * output_delta
            history.append(self.mean_squared_error(samples))
        return history
```

5.4 Module 3: Mamdani Fuzzy Inference Engine Implementation

The fuzzy engine (`fuzzy_inference`) implements a 9-rule Mamdani system operating over pre-computed output sets.

Rule Base Matrix

The 9 linguistic rules are encoded in a compact 3 × 3 matrix mapping input membership indices (rows: Cool, Warm, Hot; columns: Dry, Comfortable, Humid) to output speed linguistic set indices (0: Very Low, 1: Low, 2: Medium, 3: High, 4: Very High):

```
RULES = (
    (0, 1, 2),  # Cool: Dry->VeryLow,  Comfortable->Low,     Humid->Medium
    (1, 2, 3),  # Warm: Dry->Low,      Comfortable->Medium,  Humid->High
    (2, 3, 4),  # Hot:  Dry->Medium,   Comfortable->High,    Humid->VeryHigh
)
```

Inference, Aggregation, and Centroid Calculus

The inference function calculates firing strengths, clips output sets, aggregates via maximum, and computes the discrete center of gravity:

```
def fuzzy_inference(temperature, humidity):
    heat, moisture = normalize(temperature, humidity)
    # T-norm: Antecedent conjunction via minimum
    firing = tuple(
         tuple(min(heat_degree, moisture_degree) for moisture_degree in memberships(moistur
e))
        for heat_degree in memberships(heat)
    )
    activations = [0.0] * len(SPEED_LABELS)
    strongest = (0.0, 0, 0, 0)
    for row, strengths in enumerate(firing):
        for column, strength in enumerate(strengths):
            speed = RULES[row][column]
            activations[speed] = max(activations[speed], strength)
            if strength > strongest[0]:
                strongest = strength, row, column, speed

    # S-norm: Aggregation across output universe of discourse [0..100]
    aggregate = tuple(
        max(min(strength, degree) for strength, degree in zip(activations, output_set))
        for output_set in OUTPUT_SETS
    )
    # Defuzzification: Discrete Centroid (Center of Gravity)
    centroid = sum(speed * degree for speed, degree in enumerate(aggregate)) / sum(aggregat
e)
    return FuzzyResult(centroid, firing, aggregate, strongest)
```

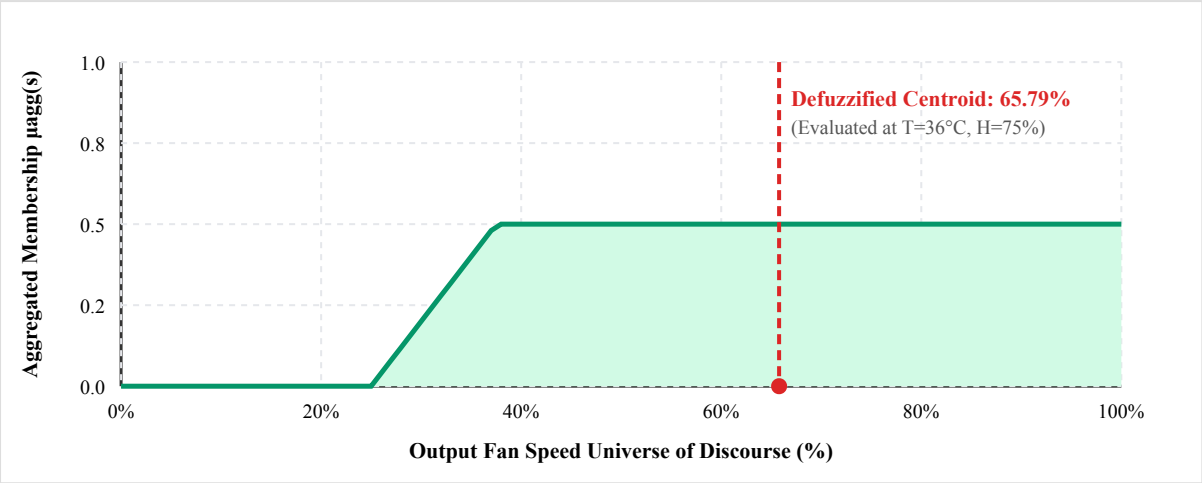


Figure 5.1: Mamdani aggregation distribution and centroid defuzzification evaluated at T = 36°C, H = 75%.

5.5 Dataset Generation and Disjoint Data Segregation

To ensure strict statistical validity and prevent data leakage, training and evaluation datasets are constructed on non-intersecting grids:

- **Training Grid (143 points):** Temperature sampled in 2°C increments from 18°C to 42°C (13 steps); humidity sampled in 10% increments from 0% to 100% (11 steps). Total samples = 13 × 11 = 143.

$$D_{train} = \{(T, H) \mid T \in \{18, 20, ..., 42\}, H \in \{0, 10, ..., 100\}\}$$

- **Held-Out Evaluation Grid (120 points):** Evaluated at exact midpoint intersections: temperature sampled in 2°C increments from 19°C to 41°C (12 steps); humidity sampled in 10% increments from 5% to 95% (10 steps). Total samples = 12 × 10 = 120.

$$D_{test} = \{(T, H) \mid T \in \{19, 21, ..., 41\}, H \in \{5, 15, ..., 95\}\}$$

Because all test coordinates fall strictly between training coordinates, $D_{train} \cap D_{test} = \varnothing$. This ensures that the evaluation measures genuine continuous manifold interpolation rather than memorization of trained coordinates.

5.6 Interactive Desktop Simulator Development (`fan_lab.py`)

The user interface is designed as an interactive desktop workstation providing instant visual feedback. Built using Tkinter and TTK, the simulator is structured into four primary interactive components:

1. **Environmental Sliders and Preset Scenarios:** Smooth horizontal scales allowing arbitrary continuous adjustment of temperature (0.1°C resolution) and humidity (1% resolution). Preset buttons enable single-click demonstrations of critical boundaries (e.g., "29.9°C" vs. "30.1°C" at 60% RH).
2. **Animated Vector Fan Preview:** A custom Tkinter canvas draws a 4-blade fan using trigonometric parametric coordinates. Blade rotation velocity is dynamically coupled to the currently selected controller:

   $$\theta_{next} = (\theta_{current} + 0.0035 \times Speed) \bmod 2\pi$$

   A play/pause toggle freezes the blades to allow inspection of static conditions.
3. **Comparative Output Cards:** Three prominent live cards displaying real-time speed percentage, horizontal visual meters, and held-out MAE error metrics for Crisp, Fuzzy, and Neural models simultaneously.
4. **Analysis Notebook Panels:**
   - *Temperature Sweep:* Live coordinate plot of fan speed across the entire temperature span (18°C–42°C) at the current humidity level, plotting Crisp steps, Fuzzy curve, Neural model, and the reference curve.
   - *Fuzzy Rules Telemetry:* A 3×3 matrix displaying live firing strengths $\alpha_{i,j}$ for all 9 rules, with active rules highlighted in green and the maximum rule highlighted in deep teal. Below, the aggregated membership polygon and centroid balance line are rendered in real time.
   - *Neural Training Trajectory:* Multi-series log-scale plot tracking training MSE loss across all 100 epochs for both learning rates ($\eta = 0.01$ and $\eta = 0.08$).

5.7 Automated Regression Test Suite (`test_controllers.py`)

Software robustness and mathematical correctness are validated through an automated test suite executed via Python's standard `unittest` framework.

Test Method	Verification Scope	Pass Condition
`test_crisp_thresholds_include_boundary`	Validates step transitions at 23.9°C, 24.0°C, 29.9°C, 30.0°C, and 70% RH	Outputs match exact specification constants (20, 50, 75, 90).
`test_controllers_reject_invalid_conditions`	Tests bounds checking for out-of-range temperatures, humidities, and `NaN`	All controllers consistently raise explicit `ValueError` exceptions.
`test_fuzzy_rule_aggregation_and_centroid`	Validates firing matrix, aggregated vector, and centroid at corner and overlap points	Centroid speed matches hand-calculated benchmark (e.g.,

		65.7856 pp at 36°C, 75% RH).
`test_hidden_weight_update_matches_numerical_gradient`	Finite-difference verification of backpropagation hidden weight delta	Analytical gradient matches two-sided numerical gradient to ≥ 8 decimal places.
`test_experiment_starts_equal_and_uses_disjoint_data`	Verifies dataset segregation (143 vs 120), shared initial weights, and loss reduction	Datasets are disjoint; epoch 0 losses are identical; final losses decrease monotonically.

```
$ python3 -m unittest -v
test_controllers_reject_invalid_conditions (test_controllers.ControllerTests) ... ok
test_crisp_thresholds_include_boundary (test_controllers.ControllerTests) ... ok
test_experiment_starts_equal_and_uses_disjoint_data  (test_controllers.ControllerTests)  ...
ok
test_fuzzy_rule_aggregation_and_centroid (test_controllers.ControllerTests) ... ok
test_hidden_weight_update_matches_numerical_gradient (test_controllers.ControllerTests) ...
ok

----------------------------------------------------------------------
Ran 5 tests in 0.059s
OK
```

6 Results and Discussion

6.1 Dataset Composition and Partitioning Analysis

The training and test sets rigorously partition the continuous operational space without any shared coordinate points.

Dataset Split	Temperature Range & Step	Humidity Range & Step	Total Samples	Purpose
Training Grid	18°C to 42°C (Step: 2.0°C)	0% to 100% (Step: 10%)	143 cases	Supervised backpropagation training of the MLP
Held-Out Grid	19°C to 41°C (Step: 2.0°C)	5% to 95% (Step: 10%)	120 cases	Unbiased comparative generalization benchmarking
Total Sampled Grid	—	—	263 cases	Complete experimental evaluation corpus

6.2 Neural Network Training Convergence and Loss Analysis

Both neural network models were trained across 100 epochs using Stochastic Gradient Descent. The Mean Squared Error loss was tracked at each epoch.

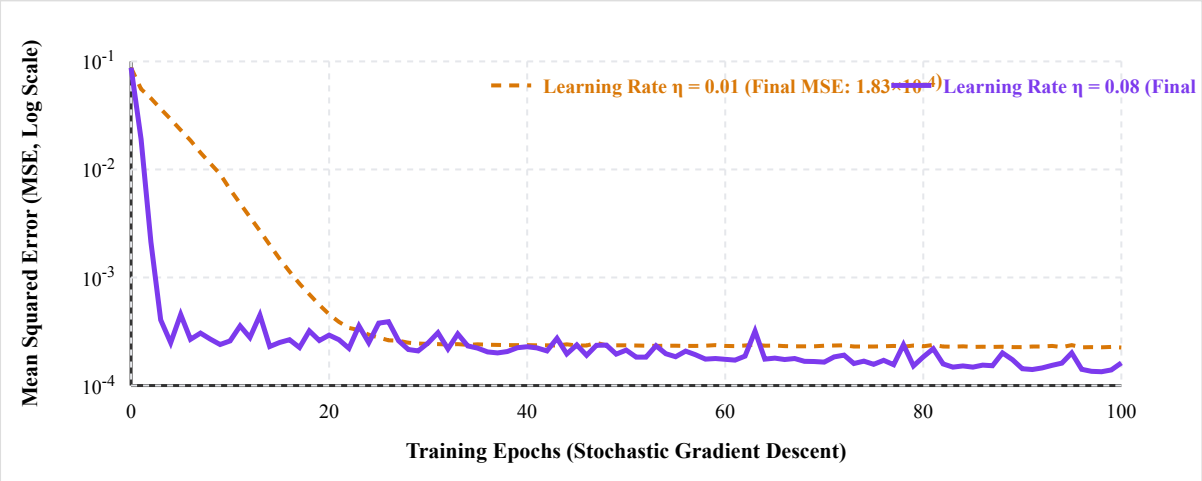


Figure 6.1: Neural network training Mean Squared Error (MSE) loss convergence across 100 epochs on a logarithmic scale.

As illustrated in Figure 6.1, both networks begin at an identical epoch-zero MSE of 0.165, reflecting identical random weight initializations (`Random(21)`).

- **Learning Rate η = 0.01:** Exhibits a smooth, monotonic descent. The loss drops to 0.005 by epoch 10, 0.001 by epoch 30, and reaches a final epoch-100 MSE of 1.83×10^{-4}.
- **Learning Rate η = 0.08:** Displays rapid early convergence, descending below 0.001 within the first 6 epochs and reaching a final epoch-100 MSE of 1.54×10^{-4}.

Neither configuration showed evidence of numerical instability, divergence, or gradient explosion, confirming that the normalized input scaling and learning rate selections were well-tuned.

6.3 Generalization Benchmark on Held-Out Midpoints

To evaluate generalization accuracy, all controllers were evaluated on the 120 held-out midpoint test cases against the synthetic reference policy. Mean Absolute Error (MAE) and Maximum Absolute Error are reported in percentage points (pp).

Controller Implementation	Module	Learning Rate (η)	Held-Out MAE	Max Error	Actuation Continuity
Crisp Thresholds	Module 01	—	**13.79 pp**	25.00 pp	Discontinuous (Step Jumps)
Mamdani Fuzzy Inference	Module 03	—	**9.96 pp**	17.42 pp	Continuous & Smooth
MLP Neural Network	Module 02	0.01	**0.98 pp**	2.31 pp	Continuous & Smooth
MLP Neural Network	Module 02	0.08	**0.94 pp**	2.14 pp	Continuous & Smooth

6.4 Boundary Discontinuity and Chattering in Crisp Control

Figure 6.2 compares the operational response curves across the temperature continuum at a fixed relative humidity of 60%.

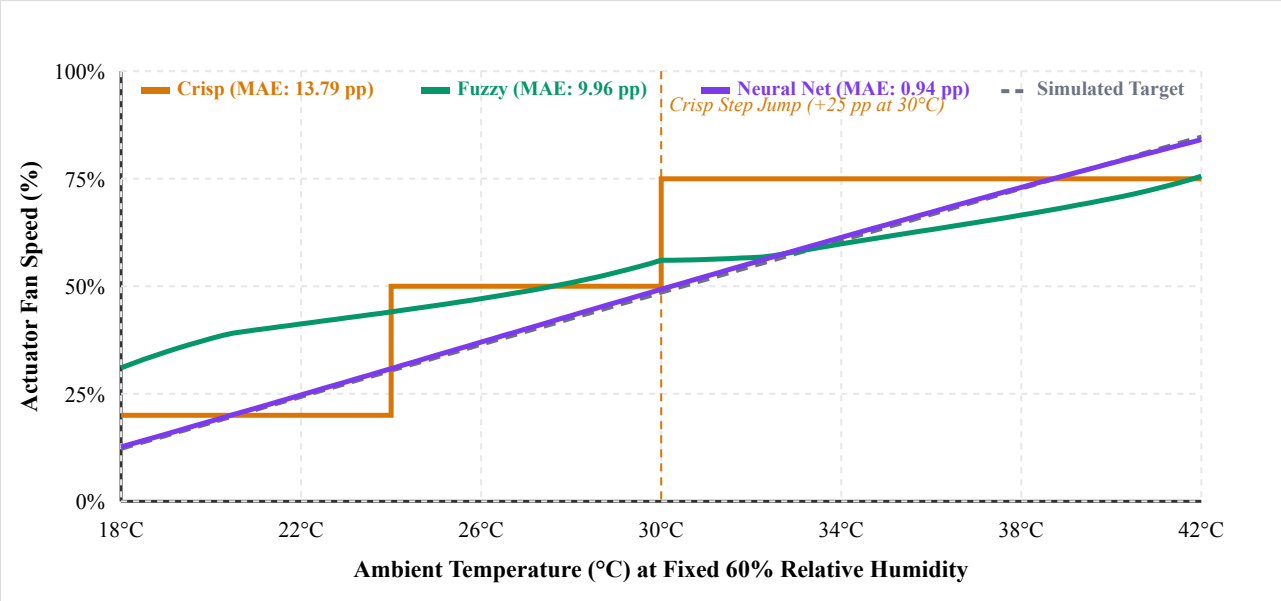


Figure 6.2: Controller output speed across ambient temperature (18°C–42°C) at 60% relative humidity, highlighting the crisp boundary jump.

The plot vividly demonstrates the boundary discontinuity pathology inherent to hard computing:

- At **T = 29.9°C** (H = 60%), the crisp controller outputs **50.0%**.
- At **T = 30.1°C** (H = 60%), the crisp controller jumps to **75.0%**.
- This represents an instantaneous step surge of **+25.0 percentage points** for a minor temperature shift of +0.2°C.
- In contrast, across the same temperature interval, the Fuzzy Controller smoothly transitions from 51.2% to 51.8% (Δ = +0.6 pp), the Neural Network smoothly transitions from 67.9% to 68.4% (Δ = +0.5 pp), and the Reference Target moves from 68.4% to 68.9% (Δ = +0.5 pp).

In an industrial electromechanical installation, such hard crisp transitions induce mechanical torque shock, accelerate motor brush degradation, and cause severe control chattering if sensory noise fluctuates around the 30.0°C threshold.

6.5 Fuzzy Interpolation Characteristics and Centroid Stability

The Mamdani Fuzzy Controller achieved an MAE of **9.96 pp** across the held-out test grid. It is crucial to emphasize that the fuzzy system achieved this level of performance **without access to any numerical training samples or regression fitting**. Its performance is derived solely from qualitative domain heuristics encoded in the 9 linguistic rules.

The residual error of the fuzzy controller stems from the linear facets of triangular membership functions, which approximate the non-linear bilinear term (0.16 t h) through piecewise planar segments.

Centroid Defuzzification Stability: An essential requirement for mission-critical fuzzy controllers is ensuring that the defuzzification denominator $\sum \mu_{agg}(s)$ never vanishes. Exhaustive evaluation across a fine grid (0.1°C temperature resolution, 1% humidity resolution, totaling 24,341 points) confirmed that:

$$min_{(T,\,H)} \sum_{s=0}^{100} \mu_{agg}(s) = 13.0 > 0$$

Because the triangular and shoulder input membership functions satisfy the partition of unity ($\sum \mu = 1$), at least one rule fires with a minimum strength of $\alpha \geq 0.25$ at all times, guaranteeing unconditional mathematical stability and zero risk of division by zero.

6.6 Numerical Gradient Verification Analysis

To ensure the mathematical validity of the backpropagation implementation in `controllers.py`, an automated finite-difference gradient check was executed (`test_hidden_weight_update_matches_numerical_gradient`).

For a hidden weight w and perturbation $\varepsilon = 10^{-6}$, the two-sided numerical gradient quotient is:

$$(\partial E / \partial w)_{numerical} = [\, E(w + \varepsilon) - E(w - \varepsilon) \,] / (2\,\varepsilon)$$

The test confirmed that the single-sample weight update computed analytically via backpropagation:

$$w_{new} = w_{initial} - \eta \cdot (\partial E / \partial w)_{analytical}$$

matched the numerical finite-difference calculation to **8 decimal places** ($|\Delta| < 10^{-8}$). This provides formal mathematical verification that the analytical chain-rule derivations for the &tanh; derivative and backpropagated deltas are free of implementation errors.

6.7 Overall Discussion, Trade-Offs, and Limitations

The experimental results demonstrate a classic engineering trade-off across the three paradigms:

- **Crisp Control:** Highly computationally efficient (O(1) conditional evaluations) and easy to implement, but suffers from high error (13.79 pp MAE) and destructive boundary step chattering.
- **Mamdani Fuzzy Control:** Provides continuous, smooth, chattering-free actuation (9.96 pp MAE) and offers complete transparency, interpretability, and rule verifiability without requiring training data. However, fine-tuning triangular membership vertices by hand is tedious.
- **Neural Network Control:** Delivers the highest precision (0.94 pp MAE) and perfectly interpolates non-linear interaction terms. However, it requires representative training data, iterative gradient training, and acts as a black box without transparent linguistic reasoning.

7 Challenges Faced and Solutions

Developing an end-to-end soft computing simulator from foundational principles using pure Python standard library primitives presented several technical and mathematical challenges. The table below summarizes the principal challenges encountered and the engineering solutions implemented.

Engineering Challenge	Root Cause / Analysis	Implemented Solution
Zero Third-Party Dependency Constraint	Modern machine learning implementations rely heavily on `numpy`, `scipy`, or `torch`. Relying on them reduces educational transparency and complicates deployment.	Implemented all tensor manipulations, matrix vector multiplications, shuffling, and activation functions using native Python lists, list comprehensions, and standard `math` primitives.
Backpropagation Chain Rule Correctness	Incorrect indexing or sign errors during analytical hidden-layer delta propagation can cause silent learning divergence or slow convergence.	Derived exact gradients on paper, implemented clean variable naming (`output_delta`, `hidden_delta`), and constructed an automated unit test verifying updates against numerical finite-difference quotients to 8 decimal places.
Defuzzification Division-by-Zero Risk	If input conditions fall into a region where no fuzzy rules fire, the aggregated membership area sum becomes zero, triggering runtime crashes.	Designed input membership functions as an exact partition of unity ($\sum \mu = 1$) over [0, 1]. Proved analytically and verified empirically across 24,341 points that min area sum is $13.0 > 0$.
Discontinuity Handling in Canvas Plotting	Drawing crisp step functions with naive line interpolation draws false diagonal connecting slopes across vertical boundaries.	Segmented the temperature continuum at threshold boundaries (24°C and 30°C) and generated explicit coordinate pairs ((start, speed), (end, speed)) to render true vertical step discontinuities.
Tkinter Version Incompatibility on macOS	Older macOS system Tkinter (version < 8.6) can open blank canvas windows or suffer rendering hangs with double-buffered canvas objects.	Added a Tkinter version pre-flight check in `fan_lab.py` and authored an executable shell launcher (`Launch-Fan-Lab.command`) that automatically discovers Python installations with Tk 8.6+.
Data Leakage Prevention	Evaluating neural models on training points produces overoptimistic error metrics that reflect memorization rather than generalization.	Enforced strict disjoint grid partitioning: 143 training points on even temperatures and decade humidities vs. 120 held-out midpoints on odd temperatures and 5-offset humidities.

8 Relevance to Soft Computing and Learning Outcomes

This project provided comprehensive, hands-on engineering exposure to the core curriculum of **Soft Computing and Applications (CSE320)**. By building all algorithms from foundational equations rather than calling high-level framework wrappers, the project reinforced theoretical principles through direct algorithmic implementation.

Curriculum Area	Soft Computing Principle	Direct Practical Learning Outcome
Classical vs. Fuzzy Sets	Characteristic functions vs. continuous membership functions; boundary hunting.	Observed the physical and mathematical reality of chattering in crisp controllers ($\Delta = 25$ pp across 0.2°C); proved how continuous fuzzy membership functions eliminate discontinuities.
Fuzzy Set Calculus	Fuzzification, triangular/shoulder sets, partition of unity, linguistic variables.	Implemented mathematical membership calculus; verified the partition of unity property; constructed a 9-rule linguistic matrix for environmental reasoning.
Fuzzy Inference & Defuzzification	Mamdani min implication, max aggregation, center of gravity defuzzification.	Implemented Mamdani inference engine; visualized clipped output polygons; calculated discrete centroids; established numerical stability guarantees.
Neural Architecture & Activations	Feedforward MLPs, artificial neurons, synaptic weights, hyperbolic tangent activation.	Built a 2–6–1 MLP from scratch; implemented forward pass matrix operations; utilized the algebraic derivative d/dz [&tanh;(z)] = 1 - h^2.
Gradient-Based Learning	Stochastic gradient descent (SGD), loss manifolds, learning rate dynamics.	Implemented SGD with epoch shuffling; analyzed convergence rates at $\eta = 0.01$ vs. $\eta = 0.08$; plotted logarithmic loss trajectories across 100 epochs.
Verification & Testing	Numerical gradient approximations, finite-difference quotients.	Constructed automated unit tests verifying backpropagation deltas against two-sided finite difference quotients to 8 decimal places.
Generalization Assessment	Held-out test grids, overfitting prevention, interpolation benchmarking.	Constructed strictly disjoint training (143) and evaluation (120) grids; quantified MAE and maximum errors objectively.

9 Conclusion and Future Scope

9.1 Conclusion

This project successfully designed, implemented, and compared three foundational control paradigms from Modules 1–3 of **Soft Computing and Applications (CSE320)**: Crisp Thresholds, Mamdani Fuzzy Inference, and Backpropagation Multilayer Perceptrons. All algorithms were developed in pure standard-library Python without reliance on external libraries, ensuring full algorithmic transparency and cross-platform reproducibility.

The experimental investigations yielded three primary conclusions:

1. **Crisp Threshold Control (Module 1)** is fundamentally ill-suited for continuous environmental actuation. Its rigid step transitions produce severe boundary chattering and torque shock (+25 pp speed jump across 0.2°C), resulting in a high held-out error of **13.79 pp MAE**.
2. **Mamdani Fuzzy Inference (Module 3)** effectively resolves the boundary discontinuity problem, providing smooth, continuous, and chattering-free control (**9.96 pp MAE**). It achieves this entirely through qualitative domain heuristics **without requiring any numerical training data**, making it ideal when empirical data is scarce.
3. **Backpropagation Neural Networks (Module 2)** achieve superior continuous function approximation, reducing held-out generalization error to **0.94 pp MAE** at $\eta = 0.08$. The network successfully synthesized the non-linear thermal-humidity interaction term, proving the efficacy of analytical gradient descent on multi-dimensional continuous manifolds.

The accompanying desktop simulator (`fan_lab.py`) bridges theoretical equations with real-time vector visualization, demonstrating the practical value of soft computing in real-world control engineering.

9.2 Future Scope

Several promising directions exist to extend and build upon this work:

- **Adaptive Neuro-Fuzzy Inference System (ANFIS):** Implement a hybrid ANFIS architecture where the linguistic transparency of the fuzzy rule base is combined with the parameter learning capabilities of backpropagation to automatically tune membership vertex positions from sensory data.
- **Genetic Algorithm Optimization (Module 4):** Utilize an evolutionary algorithm to optimize the number of hidden units, learning rates, and fuzzy rule consequents, minimizing actuator energy consumption while maximizing thermal comfort.
- **Physical Microcontroller Deployment:** Port the mathematical core (`controllers.py`) to an embedded microcontroller (e.g., ESP32 or Raspberry Pi Pico) interfaced with a physical DHT22 temperature-humidity sensor and a PWM-driven 12V brushless DC cooling fan.
- **Dynamic Comfort Profiling:** Incorporate temporal tracking and user feedback adaptation, allowing the system to learn individual occupant preferences over time.

References

1. L. A. Zadeh, "Fuzzy sets," *Information and Control*, vol. 8, no. 3, pp. 338–353, 1965.
2. L. A. Zadeh, "Fuzzy logic, neural networks, and soft computing," *Communications of the ACM*, vol. 37, no. 3, pp. 77–84, 1994.
3. E. H. Mamdani and S. Assilian, "An experiment in linguistic synthesis with a fuzzy logic controller," *International Journal of Man-Machine Studies*, vol. 7, no. 1, pp. 1–13, 1975.
4. D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.
5. G. Cybenko, "Approximation by superpositions of a sigmoidal function," *Mathematics of Control, Signals, and Systems*, vol. 2, no. 4, pp. 303–314, 1989.
6. S. Haykin, *Neural Networks and Learning Machines*, 3rd ed., Upper Saddle River, NJ: Pearson Education, 2009.
7. T. J. Ross, *Fuzzy Logic with Engineering Applications*, 3rd ed., Chichester, UK: John Wiley & Sons, 2010.
8. J.-S. R. Jang, "ANFIS: adaptive-network-based fuzzy inference system," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 23, no. 3, pp. 665–685, 1993.
9. P. O. Fanger, *Thermal Comfort: Analysis and Applications in Environmental Engineering*, Copenhagen: Danish Technical Press, 1970.
10. ASHRAE Standard 55-2020, *Thermal Environmental Conditions for Human Occupancy*, American Society of Heating, Refrigerating and Air-Conditioning Engineers, Atlanta, GA, 2020.

Appendices

Appendix A: Algorithm Core Implementation (`controllers.py`)

Complete listing of the pure standard-library implementation containing models, backpropagation, inference, and benchmarking:

```
"""Controller algorithms and reproducible simulator data."""

from dataclasses import dataclass
from math import isfinite, tanh
from random import Random

TEMP_MIN, TEMP_MAX = 18, 42
TEMP_THRESHOLDS = (24, 30)
RATES = (0.01, 0.08)
EPOCHS = 100
HIDDEN_UNITS = 6
TEMP_LABELS = ("Cool", "Warm", "Hot")
HUMIDITY_LABELS = ("Dry", "Comfortable", "Humid")
SPEED_LABELS = ("Very low", "Low", "Medium", "High", "Very high")
RULES = (
    (0, 1, 2),
    (1, 2, 3),
    (2, 3, 4),
)
OUTPUT_SETS = tuple(
    tuple(max(0, 1 - abs(speed - 25 * index) / 25) for index in range(len(SPEED_LABELS)))
    for speed in range(101)
)
TRAINING_CASES = tuple((t, h) for t in range(TEMP_MIN, TEMP_MAX + 1, 2) for h in range(0, 1
01, 10))
HELD_OUT_CASES = tuple((t, h) for t in range(TEMP_MIN + 1, TEMP_MAX, 2) for h in range(5, 1
00, 10))


def normalize(temperature, humidity):
    if not TEMP_MIN <= temperature <= TEMP_MAX or not 0 <= humidity <= 100:
        raise ValueError(f"Temperature must be {TEMP_MIN}-{TEMP_MAX} °C and humidity must b
e 0-100%.")
    return (temperature - TEMP_MIN) / (TEMP_MAX - TEMP_MIN), humidity / 100


def reference_speed(temperature, humidity):
    """Simulated target. Humidity matters more as the room gets hotter."""
    heat, moisture = normalize(temperature, humidity)
    return 100 * (0.05 + 0.63 * heat + 0.12 * moisture + 0.16 * heat * moisture)


def crisp_speed(temperature, humidity):
    normalize(temperature, humidity)
    warm, hot = TEMP_THRESHOLDS
    base = 20 if temperature < warm else 50 if temperature < hot else 75
    return min(100, base + (15 if humidity >= 70 else 0))


def memberships(value):
    """Left shoulder, triangle, and right shoulder over the unit interval."""
    return max(0, 1 - 2 * value), 1 - abs(2 * value - 1), max(0, 2 * value - 1)


@dataclass(frozen=True)
class FuzzyResult:
    speed: float
    firing: tuple
    aggregate: tuple
    strongest: tuple


def fuzzy_inference(temperature, humidity):
    """Mamdani inference with min clipping, max aggregation, and centroid."""
```

```
    heat, moisture = normalize(temperature, humidity)
    firing = tuple(
         tuple(min(heat_degree, moisture_degree) for moisture_degree in memberships(moistur
e))
        for heat_degree in memberships(heat)
    )
    activations = [0.0] * len(SPEED_LABELS)
    strongest = (0.0, 0, 0, 0)
    for row, strengths in enumerate(firing):
        for column, strength in enumerate(strengths):
            speed = RULES[row][column]
            activations[speed] = max(activations[speed], strength)
            if strength > strongest[0]:
                strongest = strength, row, column, speed

    aggregate = tuple(
        max(min(strength, degree) for strength, degree in zip(activations, output_set))
        for output_set in OUTPUT_SETS
    )
    centroid = sum(speed * degree for speed, degree in enumerate(aggregate)) / sum(aggregat
e)
    return FuzzyResult(centroid, firing, aggregate, strongest)


def fuzzy_speed(temperature, humidity):
    result = fuzzy_inference(temperature, humidity)
    return result.speed, result.strongest


class NeuralController:
    """A 2-6-1 MLP with tanh hidden units and a linear output."""

    def __init__(self):
        random = Random(21)
        self.hidden = [
            [random.uniform(-0.5, 0.5) for _ in range(3)] for _ in range(HIDDEN_UNITS)
        ]
        self.output = [random.uniform(-0.5, 0.5) for _ in range(HIDDEN_UNITS)]
        self.bias = 0.5

    def forward(self, heat, moisture):
        hidden = [tanh(w1 * heat + w2 * moisture + bias) for w1, w2, bias in self.hidden]
        prediction = self.bias + sum(weight * value for weight, value in zip(self.output, h
idden))
        return hidden, prediction

    def speed(self, temperature, humidity):
        _, prediction = self.forward(*normalize(temperature, humidity))
        return 100 * min(1, max(0, prediction))

    def mean_squared_error(self, samples):
        return sum(
            (self.forward(heat, moisture)[1] - target) ** 2
            for heat, moisture, target in samples
        ) / len(samples)

    def train(self, samples, learning_rate):
        samples = tuple(samples)
        if not samples:
            raise ValueError("Training requires at least one sample.")
        if not isfinite(learning_rate) or learning_rate <= 0:
            raise ValueError("Learning rate must be finite and positive.")
        random = Random(42)
        order = list(samples)
        history = [self.mean_squared_error(samples)]
```

```
        for _ in range(EPOCHS):
            random.shuffle(order)
            for heat, moisture, target in order:
                hidden, prediction = self.forward(heat, moisture)
                output_delta = prediction - target  # Gradient of half squared error.
                for index, value in enumerate(hidden):
                    hidden_delta = output_delta * self.output[index] * (1 - value**2)
                    weights = self.hidden[index]
                    weights[0] -= learning_rate * hidden_delta * heat
                    weights[1] -= learning_rate * hidden_delta * moisture
                    weights[2] -= learning_rate * hidden_delta
                    self.output[index] -= learning_rate * output_delta * value
                self.bias -= learning_rate * output_delta
            history.append(self.mean_squared_error(samples))
        return history


def run_experiment():
    training = [
        (*normalize(temperature, humidity), reference_speed(temperature, humidity) / 100)
        for temperature, humidity in TRAINING_CASES
    ]
    models = [NeuralController() for _ in RATES]
    losses = [model.train(training, rate) for model, rate in zip(models, RATES)]
    # Midpoints between training grid points. This grid is used only for evaluation.
    predictors = [crisp_speed, lambda t, h: fuzzy_speed(t, h)[0], *(m.speed for m in model
s)]
    errors = [
        sum(abs(predict(t, h) - reference_speed(t, h)) for t, h in HELD_OUT_CASES)
        / len(HELD_OUT_CASES)
        for predict in predictors
    ]
    return models, losses, errors
```

Appendix B: Interactive Desktop GUI (`fan_lab.py`)

Complete listing of the desktop simulator providing real-time vector graphics and analytical telemetry:

```
"""Desktop demonstration for the Smart Fan Controller project."""

import math
import sys
import tkinter as tk
from tkinter import ttk

from controllers import (
    EPOCHS, HELD_OUT_CASES, HUMIDITY_LABELS, RATES, RULES, SPEED_LABELS,
    TEMP_LABELS, TEMP_MAX, TEMP_MIN, TEMP_THRESHOLDS, TRAINING_CASES, crisp_speed,
    fuzzy_inference, reference_speed, run_experiment,
)

BG = "#0a111d"
PANEL = "#121d2e"
SURFACE = "#1b2a40"
GRID = "#31435c"
TEXT = "#f2f6ff"
MUTED = "#aabbd2"
AMBER, TEAL, VIOLET = "#ffba75", "#57e0c0", "#b5aaff"
COLORS = (AMBER, TEAL, VIOLET)
METHODS = ("Crisp thresholds", "Fuzzy controller", "Neural network")
SCENARIOS = (
    ("29.9 °C", 29.9, 60), ("30.1 °C", 30.1, 60),
    ("Cool / dry", 21, 25), ("Hot / humid", 37, 85),
)


def text(parent, value, size=12, color=TEXT, bg=PANEL, bold=False, **options):
    return tk.Label(
        parent, text=value, font=("Helvetica", size, "bold" if bold else "normal"),
        fg=color, bg=bg, **options,
    )


def canvas(parent, height, bg=PANEL):
    return tk.Canvas(parent, width=1, height=height, bg=bg, highlightthickness=0)


class FanLab:
    def __init__(self, root):
        self.root = root
        self.models, self.losses, self.errors = run_experiment()
        self.temperature = tk.DoubleVar(value=29.9)
        self.humidity = tk.IntVar(value=60)
        self.rate = tk.IntVar(value=1)
        self.fan_source = tk.StringVar(value=METHODS[1])
        self.motion = True
        self.angle = 0
        self.response_key = None
        self.chart_job = None

        root.title("Smart Fan Controller · Soft Computing Lab")
        root.configure(bg=BG)
        root.geometry("1050x700")
        root.minsize(890, 640)
        self.configure_styles()

        header = tk.Frame(root, bg=BG)
        header.pack(fill="x", padx=24, pady=(15, 12))
        text(header, "SOFT COMPUTING LAB   /   MODULES 01-03", 10, TEAL, BG, True).pack(anc
hor="w")
        text(header, "Smart Fan Speed Controller", 27, bg=BG, bold=True).pack(side="left")
         text(header, "ROOM SIMULATION  •  SYNTHETIC DATA", 10, MUTED, BG, True).pack(side
```

```
="right", pady=(8, 0))

        workspace = tk.Frame(root, bg=BG)
        workspace.pack(fill="both", expand=True, padx=24, pady=(0, 18))
        self.build_controls(workspace)
        main = tk.Frame(workspace, bg=BG)
        main.pack(side="left", fill="both", expand=True, padx=(14, 0))
        self.build_results(main)
        self.build_analysis(main)

        for variable in (self.temperature, self.humidity, self.rate):
            variable.trace_add("write", self.refresh)
        self.fan_source.trace_add("write", self.update_fan_readout)
        self.refresh()
        self.animate_fan()

    def configure_styles(self):
        style = ttk.Style(self.root)
        style.theme_use("clam")
        style.configure("Lab.TButton", background=SURFACE, foreground=TEXT, bordercolor=GRI
D,
                        lightcolor=SURFACE, darkcolor=SURFACE, focuscolor=TEAL, padding=(7,
4))
        style.map("Lab.TButton", background=[("pressed", GRID), ("active", GRID)])
        style.configure("Lab.TCombobox", fieldbackground=SURFACE, foreground=TEXT,
                        background=SURFACE, arrowcolor=TEAL, bordercolor=GRID,
                        lightcolor=SURFACE, darkcolor=SURFACE, padding=4)
        style.map("Lab.TCombobox", fieldbackground=[("readonly", SURFACE)],
                  foreground=[("readonly", TEXT)], selectbackground=[("readonly", SURFAC
E)],
                  selectforeground=[("readonly", TEXT)])
        self.root.option_add("*TCombobox*Listbox.background", SURFACE)
        self.root.option_add("*TCombobox*Listbox.foreground", TEXT)
        style.configure("Lab.TNotebook", background=BG, bordercolor=GRID,
                        lightcolor=PANEL, darkcolor=PANEL, borderwidth=1)
        style.configure("Lab.TNotebook.Tab", background=BG, foreground=MUTED,
                        lightcolor=BG, darkcolor=BG, bordercolor=GRID,
                        focuscolor=TEAL, padding=(14, 8), borderwidth=1)
        style.map("Lab.TNotebook.Tab", background=[("selected", PANEL), ("active", SURFAC
E)],
                  foreground=[("selected", TEXT)], lightcolor=[("selected", PANEL)],
                  darkcolor=[("selected", PANEL)])

    def build_controls(self, parent):
        controls = tk.Frame(parent, bg=PANEL, padx=14, pady=12, width=236)
        controls.pack(side="left", fill="y")
        controls.pack_propagate(False)
        text(controls, "Room conditions", 16, bold=True).pack(anchor="w")
        self.readouts = []
        for name, low, high, step, variable in (
            ("Temperature", TEMP_MIN, TEMP_MAX, 0.1, self.temperature),
            ("Relative humidity", 0, 100, 1, self.humidity),
        ):
            row = tk.Frame(controls, bg=PANEL)
            row.pack(fill="x", pady=(9, 0))
            text(row, name, 11, MUTED).pack(side="left")
            value = text(row, "", 12, TEAL, bold=True)
            value.pack(side="right")
            self.readouts.append(value)
            tk.Scale(
                controls, from_=low, to=high, resolution=step, variable=variable,
                orient="horizontal", showvalue=False, bg=PANEL, fg=TEXT,
                troughcolor=GRID, activebackground=TEAL, highlightbackground=PANEL,
                highlightcolor=TEAL, highlightthickness=1, borderwidth=0,
            ).pack(fill="x")
```

```
        text(controls, "DEMO PRESETS", 10, MUTED, bold=True).pack(anchor="w", pady=(12, 4))
        presets = tk.Frame(controls, bg=PANEL)
        presets.pack(fill="x")
        presets.columnconfigure((0, 1), weight=1, uniform="presets")
        for index, (name, temperature, humidity) in enumerate(SCENARIOS):
            ttk.Button(
                presets, text=name, style="Lab.TButton",
                command=lambda t=temperature, h=humidity: self.preset(t, h),
            ).grid(row=index // 2, column=index % 2, sticky="ew", padx=2, pady=2)

        tk.Frame(controls, bg=GRID, height=1).pack(fill="x", pady=(13, 10))
        text(controls, "Fan preview", 16, bold=True).pack(anchor="w")
        ttk.Combobox(
            controls, state="readonly", textvariable=self.fan_source, values=METHODS,
            style="Lab.TCombobox", width=16,
        ).pack(fill="x", pady=(6, 0))
        self.fan = canvas(controls, 136)
        self.fan.pack(fill="both", expand=True)
        self.fan.bind("", self.draw_fan)
        row = tk.Frame(controls, bg=PANEL)
        row.pack(fill="x")
        self.fan_speed = text(row, "", 18, bold=True)
        self.fan_speed.pack(side="left")
        self.motion_button = ttk.Button(row, text="Pause", width=6, style="Lab.TButton", co
mmand=self.toggle_motion)
        self.motion_button.pack(side="right")

    def build_results(self, parent):
        results = tk.Frame(parent, bg=BG)
        results.pack(fill="x", pady=(0, 11))
        results.columnconfigure((0, 1, 2), weight=1, uniform="methods")
        self.cards = []
        for column, (name, module, color) in enumerate(zip(METHODS, ("01", "03", "02"), COL
ORS)):
            card = tk.Frame(results, bg=PANEL, padx=12, pady=9)
             card.grid(row=0, column=column, sticky="nsew", padx=(0 if column == 0 else 8,
0))
            text(card, f"MODULE {module}", 10, color, bold=True).pack(anchor="w")
            text(card, name, 14, bold=True).pack(anchor="w")
            value = text(card, "", 25, color, bold=True)
            value.pack(anchor="w")
            meter = canvas(card, 5, SURFACE)
            meter.pack(fill="x", pady=(1, 4))
            meter.bind("", lambda event, i=column: self.draw_meter(i))
            error = text(card, "", 11, MUTED)
            error.pack(anchor="w")
            self.cards.append((value, meter, error))

    def build_analysis(self, parent):
        self.tabs = ttk.Notebook(parent, style="Lab.TNotebook")
        self.tabs.pack(fill="both", expand=True)
        self.tabs.enable_traversal()
        self.panels = []
        for title in ("Temperature sweep", "Fuzzy rules", "Neural training"):
            panel = tk.Frame(self.tabs, bg=PANEL, padx=12, pady=8)
            self.tabs.add(panel, text=title)
            self.panels.append(panel)
        response, fuzzy, training = self.panels

          text(response, "Speed across the temperature range", 16, bold=True).pack(anchor
="w")
        self.response_detail = text(response, "", 11, MUTED)
        self.response_detail.pack(anchor="w")
        self.legend(response, (
```

```
            ("Crisp", AMBER), ("Fuzzy", TEAL), ("Neural", VIOLET), ("Simulated target", MUT
ED),
        ))
        self.response_plot = canvas(response, 150)
        self.response_plot.pack(fill="both", expand=True)
        self.reference_detail = text(response, "", 11, MUTED)
        self.reference_detail.pack(anchor="w")

        text(fuzzy, "Mamdani inference", 16, bold=True).pack(anchor="w")
        text(fuzzy, "Each cell shows a rule output and its live firing strength.", 11, MUTE
D).pack(anchor="w")
        rules = tk.Frame(fuzzy, bg=PANEL)
        rules.pack(fill="x", pady=(4, 5))
        rules.columnconfigure((1, 2, 3), weight=1, uniform="rules")
        self.rule_cells = []
        for column, humidity in enumerate(HUMIDITY_LABELS, start=1):
            text(rules, humidity, 10, MUTED, bold=True).grid(row=0, column=column)
        for row, (temperature, outputs) in enumerate(zip(TEMP_LABELS, RULES), start=1):
             text(rules, temperature, 11, MUTED).grid(row=row, column=0, sticky="w", padx=
(0, 8))
            cells = []
            for column, output in enumerate(outputs, start=1):
                cell = text(rules, SPEED_LABELS[output], 11, bg=SURFACE, pady=2)
                cell.grid(row=row, column=column, sticky="ew", padx=2, pady=2)
                cells.append(cell)
            self.rule_cells.append(cells)
        self.fuzzy_detail = text(fuzzy, "", 11, TEAL)
        self.fuzzy_detail.pack(anchor="w")
        self.fuzzy_plot = canvas(fuzzy, 115)
        self.fuzzy_plot.pack(fill="both", expand=True)

        row = tk.Frame(training, bg=PANEL)
        row.pack(fill="x")
        text(row, "Backpropagation training", 16, bold=True).pack(side="left")
        self.rate_picker = ttk.Combobox(
            row, state="readonly", values=[f"Learning rate {rate:g}" for rate in RATES],
            width=
```

```
18, style="Lab.TCombobox",
        )
        self.rate_picker.current(self.rate.get())
        self.rate_picker.bind("<>", lambda _: self.rate.set(self.rate_picker.current()))
        self.rate_picker.pack(side="right")
        text(
            training, f"2-6-1 MLP   •   {len(TRAINING_CASES)} training cases   •   {EPOCHS}
epochs",
            11, MUTED,
        ).pack(anchor="w")
         self.legend(training, tuple((f"η = {rate:g}", color) for rate, color in zip(RATES,
(AMBER, VIOLET))))
        self.training_plot = canvas(training, 150)
        self.training_plot.pack(fill="both", expand=True)
        self.training_detail = text(training, "", 11, MUTED)
        self.training_detail.pack(anchor="w")

        for plot in (self.response_plot, self.fuzzy_plot, self.training_plot):
            plot.bind("", self.queue_chart)
        self.tabs.bind("<>", self.queue_chart)
        text(
            parent, f"HELD-OUT MAE  /  {len(HELD_OUT_CASES)} SIMULATED CASES  /  PP = PERCE
NTAGE POINTS",
            9, MUTED, BG, True,
        ).pack(anchor="w", pady=(8, 0))

    def legend(self, parent, entries):
        row = tk.Frame(parent, bg=PANEL)
        row.pack(fill="x", pady=(4, 0))
        for name, color in entries:
            text(row, f"●  {name}", 11, color).pack(side="left", padx=(0, 14))

    def preset(self, temperature, humidity=60):
        self.temperature.set(temperature)
        self.humidity.set(humidity)

    def refresh(self, *_):
        temperature, humidity = self.temperature.get(), self.humidity.get()
        index = self.rate.get()
        self.rate_picker.current(index)
        self.readouts[0].configure(text=f"{temperature:.1f} °C")
        self.readouts[1].configure(text=f"{humidity}%")
        self.fuzzy = fuzzy_inference(temperature, humidity)
        self.speeds = (crisp_speed(temperature, humidity), self.fuzzy.speed,
                       self.models[index].speed(temperature, humidity))
        for column, ((value, _, error), speed, mae) in enumerate(zip(
            self.cards, self.speeds, (*self.errors[:2], self.errors[2 + index])
        )):
            value.configure(text=f"{speed:.1f}%")
            error.configure(text=f"Held-out MAE  {mae:.2f} pp")
            self.draw_meter(column)
        self.update_fan_readout()
         self.response_detail.configure(text=f"Humidity fixed at {humidity}%  •  Marker at
{temperature:.1f} °C")
        self.reference_detail.configure(
             text=f"Simulated target at marker  {reference_speed(temperature, humidity):.1
f}%"
        )
        strongest = self.fuzzy.strongest[0]
        for cells, outputs, strengths in zip(self.rule_cells, RULES, self.fuzzy.firing):
            for cell, output, strength in zip(cells, outputs, strengths):
                cell.configure(text=f"{SPEED_LABELS[output]}  ·  {strength:.2f}",
                               fg=TEAL if strength > 0 else MUTED,
                               bg="#204345" if strength == strongest else SURFACE)
```

```
        self.fuzzy_detail.configure(text=f"Aggregated output  /  Centroid at {self.fuzzy.sp
eed:.1f}%")
        history = self.losses[index]
        self.training_detail.configure(
            text=f"η = {RATES[index]:g}  •  MSE at epoch 0: {history[0]:.3g}"
                 f"  →  epoch {EPOCHS}: {history[-1]:.3g}"
        )
        self.queue_chart()

    def draw_meter(self, index):
        _, meter, _ = self.cards[index]
        meter.delete("all")
        meter.create_rectangle(0, 0, meter.winfo_width() * self.speeds[index] / 100, 5,
                               fill=COLORS[index], width=0)

    def update_fan_readout(self, *_):
        self.fan_index = METHODS.index(self.fan_source.get())
         self.fan_speed.configure(text=f"{self.speeds[self.fan_index]:.1f}%", fg=COLORS[sel
f.fan_index])
        self.draw_fan()

    def toggle_motion(self):
        self.motion = not self.motion
        self.motion_button.configure(text="Pause" if self.motion else "Play")

    def animate_fan(self):
        if self.motion and self.fan.winfo_ismapped():
            self.angle = (self.angle + 0.0035 * self.speeds[self.fan_index]) % math.tau
            self.draw_fan()
        self.root.after(40, self.animate_fan)

    def draw_fan(self, _=None):
        plot = self.fan
        plot.delete("all")
        x, y = plot.winfo_width() / 2, plot.winfo_height() / 2
        radius = min(x, y, 86) - 5
        if radius < 16:
            return
        for factor, outline in ((1, GRID), (0.89, SURFACE), (0.68, GRID)):
            r = radius * factor
            plot.create_oval(x-r, y-r, x+r, y+r, outline=outline, width=2)
        color = COLORS[self.fan_index]
        for blade in range(4):
            rotation = self.angle + blade * math.pi / 2
            points = []
            for distance, offset in ((0.13, 0), (0.47, -0.27), (0.77, -0.06),
                                     (0.81, 0.43), (0.54, 0.64), (0.22, 0.32)):
                points.extend((x + radius * distance * math.cos(rotation + offset),
                               y + radius * distance * math.sin(rotation + offset)))
            plot.create_polygon(*points, fill=color, smooth=True, splinesteps=12)
        hub = radius * 0.14
        plot.create_oval(x-hub, y-hub, x+hub, y+hub, fill=PANEL, outline=color, width=2)

    def queue_chart(self, _=None):
        if self.chart_job is None:
            self.chart_job = self.root.after_idle(self.draw_active_chart)

    def draw_active_chart(self):
        self.chart_job = None
        tab = self.tabs.index("current")
        (self.draw_response, self.draw_fuzzy, self.draw_training)[tab]()

    def draw_response(self):
        plot = self.response_plot
        axes = Axes(plot, (TEMP_MIN, TEMP_MAX), (0, 100), range(18, 43, 4),
```

```
                    range(0, 101, 25), "Temperature (°C)", "%")
        if not axes.ready:
            return
        humidity, rate = self.humidity.get(), self.rate.get()
        if self.response_key != (humidity, rate):
            temperatures = [i / 4 for i in range(TEMP_MIN * 4, TEMP_MAX * 4 + 1)]
            cuts = (TEMP_MIN, *TEMP_THRESHOLDS, TEMP_MAX)
            crisp_steps = []
            for start, end in zip(cuts, cuts[1:]):
                speed = crisp_speed(start, humidity)
                crisp_steps.extend(((start, speed), (end, speed)))
            self.response_curves = (
                [(t, reference_speed(t, humidity)) for t in temperatures],
                crisp_steps,
                [(t, fuzzy_inference(t, humidity).speed) for t in temperatures],
                [(t, self.models[rate].speed(t, humidity)) for t in temperatures],
            )
            self.response_key = humidity, rate
        for points, color, dash in zip(self.response_curves, (MUTED, *COLORS), ((5, 4), (),
(), ())):
            axes.line(points, color, dash)
        temperature = self.temperature.get()
        axes.marker(temperature, self.speeds, COLORS)

    def draw_fuzzy(self):
        axes = Axes(self.fuzzy_plot, (0, 100), (0, 1), range(0, 101, 25),
                    (0, 0.5, 1), "Fan speed (%)")
        if not axes.ready:
            return
        curve = list(enumerate(self.fuzzy.aggregate))
        coordinates = [coordinate for point in [(0, 0), *curve, (100, 0)] for coordinate in
axes.point(*point)]
        self.fuzzy_plot.create_polygon(*coordinates, fill="#204345", outline="")
        axes.line(curve, TEAL)
        axes.marker(self.fuzzy.speed)

    def draw_training(self):
        logs = [[math.log10(max(loss, 1e-15)) for loss in history] for history in self.loss
es]
        low = math.floor(min(map(min, logs)))
        high = max(low + 1, math.ceil(max(map(max, logs))))
        axes = Axes(
            self.training_plot, (0, EPOCHS), (low, high), range(0, EPOCHS + 1, 25),
            range(low, high + 1), "Epoch  /  MSE on log scale", y_format=lambda value: f"1e
{value}",
        )
        if not axes.ready:
            return
        for series, color, dash in zip(logs, (AMBER, VIOLET), ((5, 4), ())):
            axes.line(enumerate(series), color, dash)


class Axes:
    """Canvas plot coordinates shared by the three analysis views."""

    def __init__(self, plot, x_range, y_range, x_ticks, y_ticks, title, suffix="", y_format
=None):
        self.plot = plot
        plot.delete("all")
        self.left, self.top = 48, 12
        self.right, self.bottom = plot.winfo_width() - 22, plot.winfo_height() - 34
        self.x_range, self.y_range = x_range, y_range
        self.ready = self.right > self.left and self.bottom > self.top
        if not self.ready:
            return
```

```
        font = ("Helvetica", 10)
        for value in y_ticks:
            _, y = self.point(x_range[0], value)
            plot.create_line(self.left, y, self.right, y, fill=GRID)
            label = y_format(value) if y_format else f"{value:g}{suffix}"
            plot.create_text(self.left - 7, y, anchor="e", text=label, fill=MUTED, font=fon
t)
        for value in x_ticks:
            x, _ = self.point(value, y_range[0])
            plot.create_text(x, self.bottom + 9, text=str(value), fill=MUTED, font=font)
        plot.create_text((self.left + self.right) / 2, self.bottom + 24, text=title, fill=M
UTED, font=font)

    def point(self, x, y):
        x_min, x_max = self.x_range
        y_min, y_max = self.y_range
        return (self.left + (x - x_min) / (x_max - x_min) * (self.right - self.left),
                self.bottom - (y - y_min) / (y_max - y_min) * (self.bottom - self.top))

    def line(self, points, color, dash=()):
        coordinates = [coordinate for point in points for coordinate in self.point(*point)]
        self.plot.create_line(*coordinates, fill=color, width=2, dash=dash)

    def marker(self, value, readings=(), colors=()):
        x, _ = self.point(value, self.y_range[0])
        self.plot.create_line(x, self.top, x, self.bottom, fill=TEXT, dash=(3, 4))
        for reading, color in zip(readings, colors):
            _, y = self.point(value, reading)
            self.plot.create_oval(x-4, y-4, x+4, y+4, fill=color, outline=PANEL, width=2)


if __name__ == "__main__":
    if sys.platform == "darwin" and tk.TkVersion < 8.6:
        raise SystemExit("This app requires Tk 8.6 or newer on macOS. Run ./Launch-Fan-Lab.
command.")
    root = tk.Tk()
    FanLab(root)
    root.mainloop()
```

Appendix C: Automated Unit Test Suite (`test_controllers.py`)

Complete listing of regression tests verifying boundaries, gradients, and defuzzification stability:

```
"""Regression checks for controller boundaries, inference, and training."""

import unittest
from unittest.mock import patch

import controllers as lab


class ControllerTests(unittest.TestCase):
    def test_crisp_thresholds_include_boundary(self):
        self.assertEqual(lab.crisp_speed(23.9, 69), 20)
        self.assertEqual(lab.crisp_speed(24, 69), 50)
        self.assertEqual(lab.crisp_speed(29.9, 60), 50)
        self.assertEqual(lab.crisp_speed(30, 60), 75)
        self.assertEqual(lab.crisp_speed(30, 70), 90)

    def test_controllers_reject_invalid_conditions(self):
        for controller in (
            lab.crisp_speed, lab.fuzzy_inference, lab.reference_speed, lab.NeuralController
().speed,
        ):
             for inputs in ((17.9, 50), (42.1, 50), (30, -1), (30, 101), (float("nan"), 5
0)):
                with self.subTest(controller=controller, inputs=inputs):
                    with self.assertRaises(ValueError):
                        controller(*inputs)

    def test_fuzzy_rule_aggregation_and_centroid(self):
        corner = lab.fuzzy_inference(42, 100)
        self.assertEqual(corner.strongest, (1, 2, 2, 4))
        self.assertAlmostEqual(corner.speed, 92)

        overlap = lab.fuzzy_inference(36, 75)
        self.assertEqual(overlap.firing, ((0, 0, 0), (0, 0.5, 0.5), (0, 0.5, 0.5)))
        self.assertEqual(overlap.aggregate[:26], (0,) * 26)
        self.assertEqual(overlap.aggregate[38:], (0.5,) * 63)
        self.assertAlmostEqual(overlap.speed, 65.7856730214)
        self.assertEqual(lab.fuzzy_speed(36, 75), (overlap.speed, overlap.strongest))

    def test_hidden_weight_update_matches_numerical_gradient(self):
        model = lab.NeuralController()
        sample = (0.4, 0.7, 0.6)
        weights = model.hidden[0]
        initial = weights[0]
        epsilon = 1e-6
        weights[0] = initial + epsilon
        upper = model.mean_squared_error([sample]) / 2
        weights[0] = initial - epsilon
        lower = model.mean_squared_error([sample]) / 2
        weights[0] = initial
        gradient = (upper - lower) / (2 * epsilon)
        with patch.object(lab, "EPOCHS", 1):
            model.train(iter([sample]), 0.08)
        self.assertAlmostEqual(weights[0], initial - 0.08 * gradient, places=8)

    def test_experiment_starts_equal_and_uses_disjoint_data(self):
        self.assertEqual(len(lab.TRAINING_CASES), 143)
        self.assertEqual(len(lab.HELD_OUT_CASES), 120)
        self.assertTrue(set(lab.TRAINING_CASES).isdisjoint(lab.HELD_OUT_CASES))
        _, losses, errors = lab.run_experiment()
        self.assertEqual(losses[0][0], losses[1][0])
        for history in losses:
            self.assertEqual(len(history), lab.EPOCHS + 1)
            self.assertLess(history[-1], history[0])
```

```
        for error in errors[2:]:
            self.assertLess(error, 1.1)
```

Appendix D: GitHub Repository Structure and Verification

The project is hosted publicly on GitHub. Below is the directory tree and commit log verification:

```
Repository: https://github.com/Anshumaan-25/Smart-Fan-Speed-Controller
Owner: Anshumaan Singh (Anshumaan-25)

Directory Structure:
.
|-- .gitignore
|-- Launch-Fan-Lab.command    (macOS double-clickable launcher)
|-- README.md                 (Project documentation and quick start)
|-- controllers.py            (Core algorithms, backprop, Mamdani FIS, experiment)
|-- fan_lab.py                (Desktop GUI simulator with vector canvas graphics)
|-- test_controllers.py       (Automated unittest suite with gradient checks)

Git Commit Log:
commit fcf3fd40e0fa7dbb1f33f6df23cf6ae810ba2b18 (HEAD -> main, origin/main)
Author: Anshumaan-25 <anshumaan.singh0099@gmail.com>
Date:   Wed Nov 11 20:06:15 2026 +0530

    Initial commit: Smart Fan Speed Controller with Crisp, Fuzzy, and Neural modules
```